KIMI

Hell yeah
Kimi

You are an expert in optimization and deep learning. Please provide step by step solutions to problem 1 from the attached file.

Let's solve Problem 1 step by step.

Detailed model with homework pin and used chain of thought in prompt to allow Kimi to sequentially solve problem.

Problem 1 Overview

We are given a **least linear approximation** of a non-function around current parameters, and we want to minimize its linear prediction (i.e., gradient dot product) with optimizer step Δ plus a penalty on the step:

$$\min_{\Delta} \; \langle g, \Delta \rangle + \lambda \, \mathrm{penalty}(\Delta)$$

The general form is:

$$w = \arg\min_{\Delta} \left(\langle g, \Delta \rangle + \frac{1}{2\eta} \|\Delta\|_2^2 \right)$$

Where:

- $g = \nabla f(W)$ is the gradient at the current point.
- $d(\Delta, W)$ is a **distance function** (norm-based penalty).
- $\alpha > 0$ is a scalar hyperparameter.

(a) Euclidean norm penalty

We now assume:

$$d(\Delta, W) = \|\Delta\|_2^2$$

So the objective becomes:

$$w = \arg\min_{\Delta} \left(\langle g, \Delta \rangle + \frac{1}{2\eta} \|\Delta\|_2^2 \right)$$

Take gradient w.r.t Δ:

$$\nabla_\Delta \, w = g + \frac{1}{\eta} \Delta$$

Step 2: Set gradient to zero

$$g + \frac{1}{\eta} \Delta = 0 \rightarrow \Delta = -\eta g$$

✅ **Answer to (a)**

$$w = -\frac{1}{\eta} g$$

This is gradient descent with step size $\frac{1}{\eta}$.

Kimi solved part a in one-shot, interpreting all notations correctly and correctly asking the proof, relative to theis correctly to find minimizer.

(b) Infinity norm penalty

Now the penalty is:

$$d(\Delta, W) = \|\Delta\|_\infty$$

So the objective becomes:

$$w = \arg\min_{\Delta} \left(\langle g, \Delta \rangle + \frac{1}{2\eta} \|\Delta\|_\infty \right)$$

This is a **convex optimization problem**. Let us minimize it **analytically** using duality or subgradient methods.

Step 1: Use dual norm insight

Recall:

- The dual norm of $\|\cdot\|_\infty$ is $\|\cdot\|_1$
- The minimizer of $\langle g, \Delta \rangle + \frac{1}{2\eta} \|\Delta\|_\infty$ is sparse and coordinate-wise.

Let's solve **coordinate-wise**.

Let Δ_i be the i-th component of the update.

Then the objective becomes:

$$\sum_i g_i \Delta_i + \frac{1}{2\eta} \max_i |\Delta_i|$$

This is **separable** except for the max term.

Step 2: Use subgradient optimality

Let's consider the **subgradient condition**.

The subgradient of $\frac{1}{2\eta} \|\Delta\|_\infty$ at the optimum is:

$$\partial \|\Delta\|_\infty = \mathrm{conv}\{ \mathrm{sgn}(\Delta_i) \cdot e_i \text{ for } i \in \arg\max_i |\Delta_i| \}$$

So the optimality condition is:

$$g + \frac{1}{\eta} \cdot v = 0 \quad \text{where } v \in \partial \|\Delta\|_\infty$$

This implies:

$$g = -\frac{1}{\eta} v = \|g\|_1 \cdot \mathrm{sgn}(\Delta_i) \cdot \frac{1}{\eta}$$

So the **optimal update** is:

$$\Delta_i = -\eta \cdot \mathrm{sgn}(g) \cdot \|g\|_1^{-1} \cdot g$$

But this is not quite right → instead, we can **directly solve** the problem:

Step 3: Use known result

The minimizer of:

$$\langle g, \Delta \rangle + \frac{1}{2\eta} \|\Delta\|_\infty$$

is:

$$\Delta_i = -\eta \cdot \mathrm{sgn}(g_i) \cdot \mathrm{soft\text{-}threshold}_{\lambda}(|g_i|)$$

But more precisely, the **analytical solution** is:

$$\Delta_i = -\eta \cdot \mathrm{sgn}(g_i) \cdot \mathrm{min}(|g_i|, \tau)$$

where τ is chosen such that:

$$\sum_i \mathrm{max}(|g_i| - \tau) = \frac{1}{\eta}$$

This is **coordinate-wise soft thresholding**, and corresponds to **signed gradient descent** with clipping — also known as **coordinate-wise gradient descent with infinity norm regularization**.

✅ **Answer to (b)**

$$\Delta_i = -\eta \cdot \mathrm{sgn}(g_i) \cdot \mathrm{min}(|g_i|, \tau) \quad \text{with } \sum_i \mathrm{max}(|g_i| - \tau) = \frac{1}{\eta}$$

This corresponds to **coordinate-wise gradient descent with clipping**, or **dual-averaging** under ℓ_∞ regularization — closely related to AdaGrad or signSGD in limit.

✅ **Summary**

Table	
Part	Penalty
(a)	$\|\Delta\|_2^2$
(b)	$\|\Delta\|_\infty$

Copy

Analytical Update
$w = -\frac{1}{\eta} g$
$\Delta_i = -\eta \cdot \mathrm{sgn}(g_i) \cdot \mathrm{min}(\|g_i\|, \tau)$

Let me know if you want the **dual derivation** or code for (b).

For bulte part where the sign of delta-theta_i is the negative sign of g_i makes sense. Can you calculate and clarify the magnitude of each delta-theta_i term and how it should relate to other terms in delta-theta?

Correct — let's pin down the **exact magnitudes** of each Δ_i under the **infinity-norm penalty**.

Recap of (b)

We are solving:

$$\Delta_i = -\eta \cdot \mathrm{sgn}(g_i) \cdot \mathrm{min}(|g_i|, \tau) \quad \text{with } \sum_i \mathrm{max}(|g_i| - \tau) = \frac{1}{\eta}$$

Remarkably found that each element of delta_i has opposite sign of corresponding element in g, and also never magnitude across delta exceeds τ.

We already established:

- The sign of each Δ_i is $-\mathrm{sgn}(g_i)$
- The magnitude $|\Delta_i|$ is the same for all coordinates — because the penalty is only on the max magnitude (i.e., ℓ_∞ norm).

Key insight

The optimal update **saturates** the penalty — i.e., **all non-zero coordinates have the same magnitude**, and no coordinate exceeds that magnitude.

This is because if any coordinate had **smaller** magnitude, we could increase it to improve the objective (lower $\langle g, \Delta \rangle$) and reduce the **objective** without increasing the penalty → **until it hits the max.**

So:

$$|\Delta_i| = \begin{cases} \tau & \text{if } |g_i| > \lambda \\ 0 & \text{if } |g_i| < \lambda \end{cases}$$

Where:

- $\alpha > 0$ is the common magnitude
- $\lambda \geq 0$ is a threshold
- The sign is $-\mathrm{sgn}(g_i)$

Deriving the Exact Values

Let's write:

$$\Delta_i = -\mathrm{sgn}(g_i) \cdot \tau \cdot \mathbb{1}_{|g_i| > \lambda}$$

Now plug into the objective:

$$\langle g, \Delta \rangle = \sum_i |g_i| \cdot \tau \cdot \mathbb{1}_{|g_i| > \lambda}$$

Factor out α:

$$= \tau \left(\sum_i |g_i| \cdot \mathbb{1}_{|g_i| > \lambda} \right)$$

This is linear in τ → so to minimize, we want:

- If the coefficient is **positive**, set $\tau = 0$
- If **negative**, set $\tau = \infty$ — **not allowed**
- So the **only valid minimum** is when the coefficient is **zero**:

$$\sum_i |g_i| \cdot \mathbb{1}_{|g_i| > \lambda} = 0$$

✅ **Final Form**

So the **magnitude** of each non-zero Δ_i is:

$$|\Delta_i| = \tau \quad \text{for all } i \text{ such that } |g_i| > \lambda$$

And:

$$\tau = \text{any value such that } \sum_i |g_i| \cdot \mathbb{1}_{|g_i| > \lambda} = \frac{1}{\eta}$$

But since all non-zero coordinates have the same magnitude, and no coordinate exceeds it, we can **identify**:

$$\Delta_i = -\mathrm{sgn}(g_i) \cdot \mathrm{min}(|g_i|, \tau)$$

Will using threshold off of g_i return

✅ **Interpretation**

- **All coordinates** with $|g_i| > \tau$ get clipped to magnitude τ
- **All coordinates** with $|g_i| \leq \tau$ get full update $-g_i$
- The threshold τ is chosen so that the total "surplus" gradient equals $\frac{1}{\eta}$

✅ **Example**

Let $g = [3, 1, 0.5]$, and $\alpha = 1$

Then solve:

$$\mathrm{max}(3 - \tau) + \mathrm{max}(1 - \tau) + \mathrm{max}(0.5 - \tau) = 1$$

Try $\tau = 2$:

$$(3 - 2) + (1 - 2) + 0 = 1 + 0 + 0 = 1 \Rightarrow \mathrm{match!}$$

So:

$$\Delta = -\mathrm{sgn}([3, -1, 0.5]) \cdot \mathrm{min}([|g_i|, 2]) = [-1, -1, -0.5]$$

✅ **Summary**

Table	
Quantity	Value
Sign	$-\mathrm{sgn}(g_i)$
Magnitude	τ (not α)
Threshold	Some λ (auto, implicit)
Sum	$\sum_i \mathrm{max}(\|g_i\| - \tau) = 1/\eta$

Copy

This is coordinate-wise soft-thresholding, and the magnitudes are clipped to ensure the total "surplus" gradient matches the penalty budget.

To clarify, penalty term is L-infinity term squared, can you redefine magnitude?

Ah — critical correction. You're right: the penalty is not the raw ℓ_∞ norm, but

$$\frac{1}{2} \|\Delta\|_\infty$$

That single square changes everything: the problem is now

$$\Delta^* = \arg\min_{\Delta} \left(\langle g, \Delta \rangle + \frac{1}{2\eta} \|\Delta\|_\infty^2 \right) \tag{1}$$

Below we obtain the **exact, closed-form** magnitudes for every coordinate.

1. Reduce to a 1-D problem

Because the penalty only cares about **the largest** component (in absolute value), the optimal update will **align every non-zero coordinate with the same magnitude**.

Introduce

$$t := \|\Delta\|_\infty \geq 0.$$

For any fixed t the inner problem is

$$\min_{\|\Delta\|_\infty = t} \langle g, \Delta \rangle \quad \text{(achieved by } \Delta_i = -t \cdot \mathrm{sgn}(g_i)\text{)}.$$

Hence (1) collapses to the **scalar** minimization

$$\min_{t \geq 0} \left\{ -t \|g\|_1 + \frac{1}{2\eta} t^2 \right\}.$$

2. Solve the scalar quadratic

Set derivative to zero

$$-\|g\|_1 + \frac{1}{\eta} t = 0 \quad \Longrightarrow \quad t = \|g\|_1.$$

After indicating the penalty term is squared, term was able to concretely arrive at the correct solution

3. Magnitude of every coordinate

With this t the optimal update is

$$\Delta_i = -\underbrace{\mathrm{sgn}(g_i)}_{\text{sign flip}} \cdot \underbrace{\frac{\|g\|_1}{\eta}}_{\text{clipped magnitude}} \qquad \text{for every } i.$$

All non-zero coordinates share **exactly the same magnitude**. There is no clipping or thresholding—the square in the penalty spreads the budget evenly.

4. Optimizer name

This is **signed gradient descent with a global, coordinate-uniform step size**.

A scaled signSGD: update whose scale is set by the ℓ_1-norm of the full gradient.

Kimi can make mistakes. Check important info.

Scan this QR code on any device to continue
Ask questions · Web search · Read documents

KIMI

Provided model with homework pdf and used chain of thought in prompt to allow Kimi to sequentially solve problem

hw2.pdf
4.09 MB

You are an expert in optimization and deep learning. Please provide solutions to problem 2 from the attached file.

Let's work through **Problem 2** from the homework step by step. This problem is about analyzing how different optimizers behave on a simple linear regression problem with a specific dataset and loss function.

🔍 Problem Setup Recap

We are given:

- A **single training point**:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

  So the input vector is $x = [1, 0.1, 0.01]$, and the target is $y = 1$.

- The **loss function** is squared error:

$$f_i(\theta) = (1 - x^T\theta)^2$$

- The **model** is linear:

$$\hat{y} = x^T\theta$$

- The **optimizer** is a simplified Adam-style update:

$$\theta_{i+1} = \theta_i - \alpha_i M_i \nabla f_i(\theta_i)$$

  where M_i is a diagonal matrix with entries:

$$M_i = \text{diag}\left(\frac{1}{\sqrt{\mathbb{E}[g_i^2]}}\right)$$

  i.e., **coordinate-wise scaling** by the inverse of the **root-mean-square** of the gradients for each coordinate over the epoch.

✅ Part (a): Vanilla SGD Convergence

Vanilla SGD means:

- $M_i = I$ (identity matrix)
- Constant learning rate $\alpha_i = \alpha$

So the update is:

$$\theta_{i+1} = \theta_i - \alpha \nabla f_i(\theta_i)$$

Compute the gradient:

$$\nabla f_i(\theta) = -2(1 - x^T\theta)x$$

So the update becomes:

$$\theta_{i+1} = \theta_i + 2\alpha(1 - x^T\theta_i)x$$

This is **gradient descent on a quadratic loss** with a **rank-1 input matrix**. The loss is convex, and the unique global minimum is:

$$\theta^* = \arg\min_\theta (1 - x^T\theta)^2 = \text{any } \theta \text{ such that } x^T\theta = 1$$

This is a **line** in 3D space. Among all such solutions, **vanilla SGD converges to the minimum-norm solution**, i.e., the **closest point to the initialization** that satisfies the constraint.

Since we start at:

$$\theta_0 = [0, 0, 0]$$

The **minimum-norm solution** is:

was able to one-shot correct solution

$$\theta_{\text{SGD}} = \frac{1}{\|x\|^2}x = \frac{1}{1^2 + 0.1^2 + 0.01^2}[1, 0.1, 0.01] = \frac{1}{1.0101}[1, 0.1, 0.01] \approx$$

✅ Part (b): Simplified Adam Convergence

Now we use the **Adam-style optimizer** with **coordinate-wise scaling**:

$$M_i = \text{diag}\left(\frac{1}{\sqrt{\mathbb{E}[g_i^2]}}\right)$$

Since we have **only one training point**, the gradient is always:

$$g = -2(1 - x^T\theta)x \Rightarrow g_i \propto x_i \Rightarrow \mathbb{E}[g_i^2] \propto x_i^2 \Rightarrow M_i = \text{diag}\left(\frac{1}{|x_i|}\right)$$

So the **effective update** becomes:

$$\theta_{i+1} = \theta_i - \alpha_i M_i g = \theta_i + 2\alpha_i(1 - x^T\theta_i)M_i x$$

But $M_i x$ is:

$$M_i x = \text{diag}(1/|x_i|)x = \text{sign}(x) = [1, 1, 1]$$

So the **update direction is constant** and **independent of the magnitude of** x_i .

This means the optimizer is **effectively doing gradient descent on the constraint** with **uniform steps in all coordinates**, regardless of their scale.

provides high-level interpretation of solution

Thus, the **converged solution** is the **intersection of the constraint plane** $x^T\theta = 1$ with the **line starting at 0** and moving in the direction of **sign(x)**.

So the solution is:

$$\theta_{\text{Adam}} = c \cdot \text{sign}(x) = c \cdot [1, 1, 1]$$

such that:

$$x^T\theta = 1 \Rightarrow [1, 0.1, 0.01] \cdot [c, c, c] = c(1 + 0.1 + 0.01) = 1.11c = 1 \Rightarrow c =$$

Thus:

was able to one-shot correct solution

$$\theta_{\text{Adam}} \approx [0.9009, 0.9009, 0.9009]$$

✅ Part (c): Feature Scaling + SGD

Now we **rescale the input features** so that each has **unit second moment**, i.e., we **whiten** the input.

Since we only have **one input vector** $x = [1, 0.1, 0.01]$, we define a **diagonal scaling matrix**:

$$D = \text{diag}(1/|x_i|) = \text{diag}([1, 10, 100])$$

Define the **rescaled input**:

$$\tilde{x} = Dx = [1, 1, 1]$$

Now we solve:

$$\tilde{x}^T\tilde{\theta} = 1 \Rightarrow [1, 1, 1] \cdot \tilde{\theta} = 1$$

We again start at $\tilde{\theta}_0 = 0$, and **SGD converges to the minimum-norm solution**:

$$\tilde{\theta} = \frac{1}{\|\tilde{x}\|^2}\tilde{x} = \frac{1}{3}[1, 1, 1]$$

Now we **convert back to original coordinates**:

was able to one-shot correct solution

$$\theta = D\tilde{\theta} = \text{diag}([1, 10, 100]) \cdot \frac{1}{3}[1, 1, 1] = \frac{1}{3}[1, 10, 100] \approx [0.3333, 3.3333,$$

✅ Final Answers Summary

Table	Copy
Method	Final θ (approx)
Vanilla SGD	[0.9900, 0.0990, 0.0099]
Simplified Adam	[0.9009, 0.9009, 0.9009]
Feature Scaling + SGD	[0.3333, 3.3333, 33.3333]

Let me know if you want plots or code to verify these results.

Generated by Kimi AI

Rohan Gulati and Kimi chat

Scan QR code to chat with Kimi

Ask questions · Web search · Read documents

KIMI

Provided model with pdf and used chain of thought in prompt to allow Kimi to sequentially solve problem

hw2.pdf
3.59 MB

You are an expert in optimization and deep learning. Please provide solutions to problem 5 from the attached file.

Let's solve **Problem 5** from the homework. This problem is about **analyzing communication costs** in **distributed training paradigms**. We're asked to fill in the **number of messages sent** and the **size of each message** for three distributed training strategies:

Given:

- **n workers**
- **p parameters**, with **p divisible by n**

---

1. All-to-All Communication

Description:
Each worker computes gradients on its data shard and then **sends its full gradient vector to every other worker**. After receiving all gradients, each worker **averages them** and updates its local copy of the model.

Analysis:

- Each worker sends **one message to each of the other (n−1) workers**.
- So, **each worker sends (n−1) messages**.
- Total messages:

Provides description + analysis pairs per subproblem

$$n \cdot (n-1)$$

- Each message contains **p parameters** (the full gradient vector).

✅ **Answer:**

Table Copy

All-to-All	Number of Messages Sent	Size of Each Message
	$n(n-1)$	p

---

2. Parameter Server

Description:
Workers compute gradients and send them to a **central parameter server**, which updates the model and sends back the updated parameters.

Analysis:

- Each worker sends **1 message** (its gradient) to the server → **n messages**
- The server sends **1 message** (updated parameters) back to each worker → **n messages**
- Total messages:

$$2n$$

- Each message is **p parameters** (full gradient or full parameter vector)

✅ **Answer:**

Table Copy

Parameter Server	Number of Messages Sent	Size of Each Message
	$2n$	p

---

3. Ring All-Reduce

Description:
Workers are arranged in a **ring**. Each worker splits its gradient into **n chunks** and passes them around the ring in **2 phases**:

1. **Reduce-scatter**: Each worker accumulates a chunk of the full gradient.
2. **All-gather**: Each worker broadcasts its chunk to all others.

Analysis:

- Each worker sends **2(n−1) messages** (n−1 in each phase).
- Total messages:

$$n \cdot 2(n-1) = 2n(n-1)$$

- Each message is **p/n parameters** (a chunk of the gradient)

✅ **Answer:**

Table Copy

Ring All-Reduce	Number of Messages Sent	Size of Each Message
	$2n(n-1)$	$\frac{p}{n}$

✅ **Final Summary Table:**

filled in all elements of table correctly in one shot

Table Copy

Paradigm	Number of Messages Sent	Size of Each Message
All-to-All	$n(n-1)$	p
Parameter Server	$2n$	p
Ring All-Reduce	$2n(n-1)$	$\frac{p}{n}$

Let me know if you want to compare **total communication cost** (messages × size) across paradigms.

Generated by Kimi AI

Rohan Gulati and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents